

JCOP Remote Communication Protocol (JRCP) v2.2

Protocol specification for communicating with remote JCOP devices

Rev. 2.8 — Jul 21, 2017

Application note

Document information

Info	Content
Keywords	JRCP, remote communication, JCOP Tools, JCShell, JCOP Simulator,
Abstract	This document describes the so called JCOP Remote Communication Protocol (JRCP) for communicating with a JCOP product connected to a remote server device based on TCP. JRCP is supported by JCShell and JCOP Simulators being part of the JCOP Tools.



JCOP Remote Communication Protocol (JRCP) v2.2

NXP Semiconductors

Protocol Specification

Revision history

Rev	Date	Description
0.9	Oct 28, 2011	Initial Version
2.0	Feb 19, 2016	Extended command format to handle extended-length APDUs, event handling, a re-definition of the communication to detect time-outs, a version and feature request discovery
2.1	Apr 12, 2016	Clarifications of the protocol, requests are now mapped to responses of the same MTY, unified generic error codes, added sub-MTYs for distinction of message variants, definition of extensions and device-specific commands
2.2	Apr 20, 2016	Further clarifications, fix of a formatting issue in Section 4.5, coding of the timing information adjusted, specified error behavior for cold and warm reset, handling of HCI events, moved all example communication to the corresponding examples section
2.3	May 18, 2016	Allow all NADs for the feature control request, clarification of the feature control request, proper use of sub-MTYs for feature control requests, changed example communication to use the new message format
2.4	Jan 13, 2017	Minor clarifications regarding terminal information
2.5	Feb 20, 2017	Add a version agreement between client and server, separate the JRCP version from the revision of the JRCP specification, minor textual modifications. New JRCP version 2.1: Added "Set IO Pin (MTY 0Eh)", limit maximum overall message length to FFFFFFFFh
2.6	Mar 17, 2017	The controller configuration has been extended to allow for fixed (pre-defined) nodes. JRCP version increased to 2.2: Added "Send session ID" (MTY FFh, sub-MTY 0005h).
2.7	Apr 18, 2017	Corrections of length usage in requests and responses. Added a new bit definition if timing information is requested and responded in table 17 of section 4.12.
2.8	Jul 21, 2017	Correction of time base of prepare tearing. Clarification of used data types, supported value ranges and major textual modifications.

Contact information

For more information, please visit: <http://www.nxp.com>

For sales office addresses, please send an email to: salesaddresses@nxp.com

1. Introduction

This document specifies the JCOP Remote Communication Protocol (JRCP). JRCP offers a set of commands to communicate with JCOP products connected remotely. Besides offering an interface to communicate with JCOP, JRCP allows to communicate regular Application Protocol Data Units (APDUs) as defined by ISO/IEC 7816-4 with non JCOP targets and it also provides commands to interact with the devices (e.g., to send and receive status/error information) remotely. A powerful set of JCOP Tools, i.e. the JCShell, the JCOP simulator and the JRCP Server, do implement JRCP.

1.1 Overview and Scope

Section 2 introduces the concept of communicating from a client with a remote server, the involved system components, as well as how information can be exchanged.

Section 3 specifies the protocol including the message format and the device management including the event handling.

Section 4 defines all specified commands.

Section 5 lists obsolete features and commands that have been part of previous JRCP versions.

Section 6 provides an example of a client and server session including a sequence diagram and message exchanges based on JCShell and a remote JCOP product.

It is not the aim of this application note to describe details about how to exactly implement a remote server, but rather to provide a protocol specification defining the behavior of an implementation thereof.

1.2 Intended Audience

This document guides (software) developers and architects who develop software for devices containing a JCOP product.

1.3 Ordering JCOP Tools

The JCOP Tools, containing components implementing this specification, are made available to customers using products containing JCOP. As the JCOP Tools contain product-specific implementation details, they are provided under the same restrictions as products containing JCOP. So rules for export control and confidentiality (NDA) apply in addition to software license conditions.

1.4 Nomenclature

During this document, the key words “MUST”, “MUST NOT”, “REQUIRED”, “SHALL”, “SHALL NOT”, “SHOULD”, “SHOULD NOT”, “RECOMMENDED”, “MAY”, and “OPTIONAL” in this document are to be interpreted as described in [1]. Additionally, the term “OBSOLETE” describes features that have been part of previous versions of the specification but have been removed and SHALL not be supported in current

JCOP Remote Communication Protocol (JRCP) v2.2

implementations. “DEPRECATED” denotes a feature that should be supported by server and client implementations but SHALL be obsoleted in future revisions of the specification.

1.5 Notation

The notation of values and numbers used in this specification are shown in decimal form, i.e. 15 (fifteen), hexadecimal form, i.e. Fh (fifteen) or binary form, i.e. 1111b (fifteen) unless explicitly stated otherwise.

The concatenation of value used into a combined data field used in this specification are indicated by using the character “+”, i.e. 12h+34h indicates a combined data field equal to 1234h.

A not existent value of an empty field is indicated by the character “-”.

Optional fields are indicated by the usage of squared bracket open and close, i.e. “[TR]” indicates that the parameter TR is optional.

To indicate a choice between two fields or option the logical operator OR is indicated by using the character “|”, i.e. “[TR] | [TR+TC]” indicates that a given field might be either empty, contain TR or a concatenation of TR and TC.

Units used in this specification are shown in squared bracket open and close, i.e. [ms] for milliseconds or [us] for microseconds.

1.6 Reserved for Future Use

The term Reserved for Future Use (RFU) defines the following specific behavior of implementations according to this specification:

- Fields defined as RFU SHALL be always sent as zero unless explicitly stated otherwise.
- Values defined as RFU SHALL NOT be send.
- Commands defined as RFU SHALL be responded as defined in section 4.11.
- Unless explicitly stated otherwise, values defined as RFU SHALL be disregarded by the receiver and the receiver SHALL keep the same interpretation of all other fields.

1.7 Not Allowed

The term Not Allowed defines the following specific behavior of implementations according to this specification:

- Values defined as Not Allowed SHALL NOT be sent.
- When a Not Allowed value is received the implementation SHALL abort the command execution and return the according status code defined for the command.

JCOP Remote Communication Protocol (JRCP) v2.2

2. JRCP Overview

JRCP specifies a communication protocol for connecting a client running on a host with a remote server. It uses TCP as actual connectivity layer which is for typically implementations addressed using socket services. On the remote server a controller manages one or more devices. These devices connect to targets using various protocols as needed. Such targets constitute the end point of communication and so the client is able to exchange commands with the target(s).

A device might be a smart card reader, an NFC controller or any logical or physical implementation connecting to a target.

A target might be a Secure Element (SE), an emulation or simulation therefore as well as any other implementation that is able to process APDUs.

Figure 1 shows an example setup where a client computer is connected over TCP to the remote server. The remote server implements a controller that translates, executes and also dispatches JRCP messages (commands) being forwarded to the connected physical device(s) or implemented logical device(s). Physical devices interface to targets using dedicated communication protocols and logical devices interface to targets via SW APIs. The JCOP Simulator supports JRCP directly and so acts as a remote server implementing a controller, logical devices and targets, on its own. For other (physical) devices to get connected via JRCP the remote server needs to implement a so called JRCP Server that implements a controller interfacing between the JRCP and the actual (physical) devices supported.

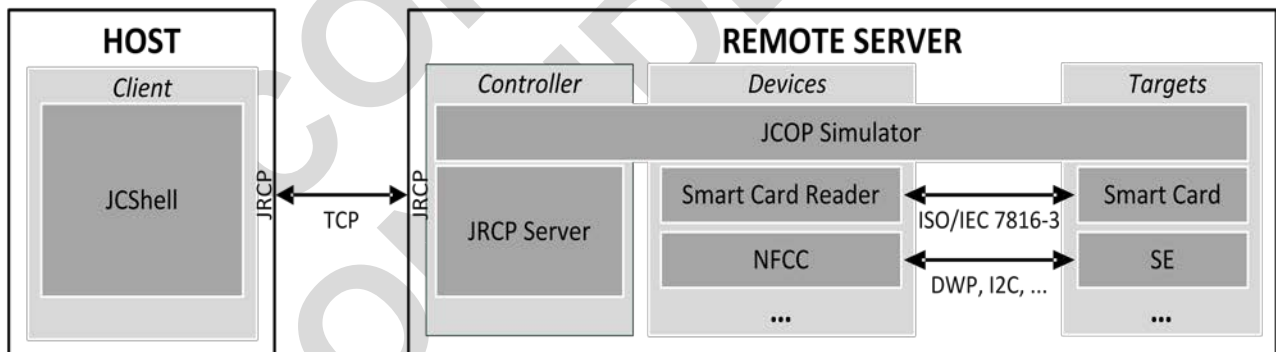


Fig 1. Client and Server Device Architecture

3. JRCP Definition

The JRCP defines commands that are coded in well-defined message formats. A command exchange is split between so called *requests* if they originate from the client, and *responses* if issued by the remote controller. A special type of response exists to signal the status of a last command executed. These status messages are described in Section 4.11. The client is the master of the protocol, the remote server (as a slave) may not initiate a communication. A server, implementing version 2.0 or higher (never in the sense or more recent versions) of this specification, is forced to answer all commands with either a valid

JCOP Remote Communication Protocol (JRCP) v2.2

response to the request, or with a status code indicating an error. So, the server SHALL not silently ignore a command.

Though, a client implementation, in accordance with this specification, SHALL include a time-out to detect unresponsive servers as well as lost connections after which the connection SHALL be terminated on the client side.

In case an obsolete, invalid, or unsupported command has been received by a remote controller, a response of the same MTY with the generic status code “Unsupported command received” (see section 04.23 “Generic Status Codes”) SHALL be returned.

The communication is categorized into the following main tasks:

- **Version discovery** is performed to determine the protocol version supported by server and client.
- **Server configuration** may optionally be performed to connect to devices and to configure protocols used to communicate with targets.
- **Device binding** encompasses obtaining a list of connected devices and targets, and reporting them to the client.
- **Target activation** describes initializing a connection between server and the target.
- **Command exchange** is the intended scope of the protocol and encompasses all interaction of the client with the target, the device, or the server after the initial configuration.

3.1 Data types and value ranges

All names of data types referenced in this specification refer to ANSI C99 defined data types like short for (values of) 16 bit (size), long for 32 bit and long long for 64 bit as well as the related unsigned representations thereof. The according maximum value range is taken from this definition accordingly, i.e. USHORT_MIN to USHORT_MAX from 0 to 65535 (FFFFh) for unsigned short, if not explicitly specified otherwise.

3.2 Endianness

All value fields defined in this specification consisting of a sequential order of bytes are represented in big-endian format.

3.3 Version Discovery

During version discovery, the main objective is to determine the version of the implemented JRCP, most notably whether a version up to 0.9 or 2.0 and above is supported. So during version discovery and when implementations indicating versions of up to 0.9 are addressed only commands using the legacy message format SHALL be used. Only when implementations indicating versions of 2.0 or above are addressed the extended message format with its additional features are available. This is performed in two steps: first, the “*Terminal Information*” request (Section 4.5) is sent in the legacy format. The ASCII encoded response (in both formats) either contains a string describing the remote terminal for versions below 2.0, or a fixed identifier for implementations that support the 2.0 version

JCOP Remote Communication Protocol (JRCP) v2.2

or higher. If a legacy server has been identified, the client SHALL continue using the 0.9 protocol. Otherwise, the “*Feature Control Request*” command (Section 4.22) SHALL be used to determine the exact version of the protocol in a second step and so to detect which commands and options can be addressed to the remote server based on the specific version returned in the response. SHALL

3.3.1 Version Agreement

Once the client has identified the server version information it SHALL continue with a version number agreement to identify if the local implementation of the client is compatible with the remote implementation of the server. This agreement SHALL be implemented applying the following rules:

- In the case where both the major and the minor version numbers of both client and server are identical then the server and the client are fully compatible with each other.
- In the case where the major version numbers are identical but the minor version numbers are not identical then version agreement SHALL be achieved. The agreed minor version number of features to be used SHALL then be the lower version number.
- In the case where the major version number of the client is higher than the version number of the server, the server may continue but the client may not be backwards compatible and so the communication may fail.
- In the case where the major version number of the client is lower than the version number of the server, the server may continue when it does implement and support the lower version number feature set. In such case the client SHALL only use commands that are defined in the specification identified by the major and minor version number returned by server. This version matched command usage SHALL be strictly followed by the client, even if it implements a higher version of this specification. If a client is unable to implement a command handling in accordance to the version returned it SHALL not establish a communication beyond version discovery.

3.3.2 JRCP Version

A client as well as a remote server implementing this revision of the specification SHALL indicate a JRCP version 2.2 meaning that the value of two for the major version and the value of two for the minor version SHALL be used.

The major version number corresponds to a major feature set and SHALL only be incremented if the new revision of this specification breaks JRCP compatibility with the previous version. Protocol versions that differ only in the minor version number are guaranteed to be backwards compatible.

Note: The versioning of the revision of this specification document is independently managed from the JRCP version. So the version returned by remote server of an implementation does not contain the revision of this document but the JRCP version as defined in this section.

3.4 Server Configuration

The server configuration is only supported for post-0.9 versions. This typically includes device-specific communication (Section 4.20 “*Device-specific Extensions*”) to configure the server itself or connected devices.

3.5 Device Binding

The device binding is required to ensure a stable mapping between node addresses and devices on the remote server. The “Controller Configuration” commands (Section 4.21) SHALL be used to obtain a list of devices connected to the server and to initialize connections between the server, its devices, and the targets.

3.6 Target Activation

The activation of targets includes “Wait for Card” commands and resetting.

3.7 Command Exchange

The final stage describes the command exchange, the communication, between the client and the target, the device, or the server on the remote side. The extended message format supporting all extended-length APDUs and proper addressing of multiple devices SHALL be used. The “*Feature Control Request*” command is mandatory to query whether a specific command is supported before it is used for the first time.

3.8 Mapping Between Devices and NADs

The JRCP allows to connect to multiple devices and so to multiple targets from a single client. To separate the requests and responses to different devices into logical channels the so called Node Address (NAD) byte is used.

To make use of a dedicated NAD and to know which actual device is identified by each NAD being used needs to be assigned to one specific device.

A current mapping of the NADs (00h – FEh) to the devices can be frozen. As long as this freeze is in place, newly connected devices SHALL not be reported when requesting a list of connected devices (see Section 4.21). Likewise, if a device is removed during this time, the NAD SHALL become orphaned and all communication attempts SHALL result in the generic error “Card Removed” (Section 4.23).

In both cases, the mapping SHALL be released and a new one is created. As releasing the freeze might invalidate existing connections, these are terminated on release. The commands to get, freeze, and release a mapping between connected devices and NADs is given in Section 4.21. The set of node addresses is split into two groups: the range of NAD 00h – 7Fh is reserved for nodes that are mapped to devices by the controller and may not be changed. The upper range with NADs 80h – FEh may be assigned to devices by the client configuring the controller in the remote server.

JCOP Remote Communication Protocol (JRCP) v2.2

The controller may not support dynamic allocation of NADs. In this case, only an at server startup/configuration predefined set of node addresses in the lower range is reported when requesting a list of devices.

The controller is always mapped to the fixed NAD value of FFh and so the client can always address the controller directly. This mapping is not impacted by the mechanism described above.

The following figure describes the communication in case of events. If an event occurred on the device, it is collected in a queue on the server. Then, the next response to the client is intercepted and replaced by an event message. The event is stored on the client, and a continuation request is issued towards the server. This is repeated until all events are reported to the client, at which point the response to the original command is sent. The Send Data command is chosen as an example here, but the event handling applies to all commands.

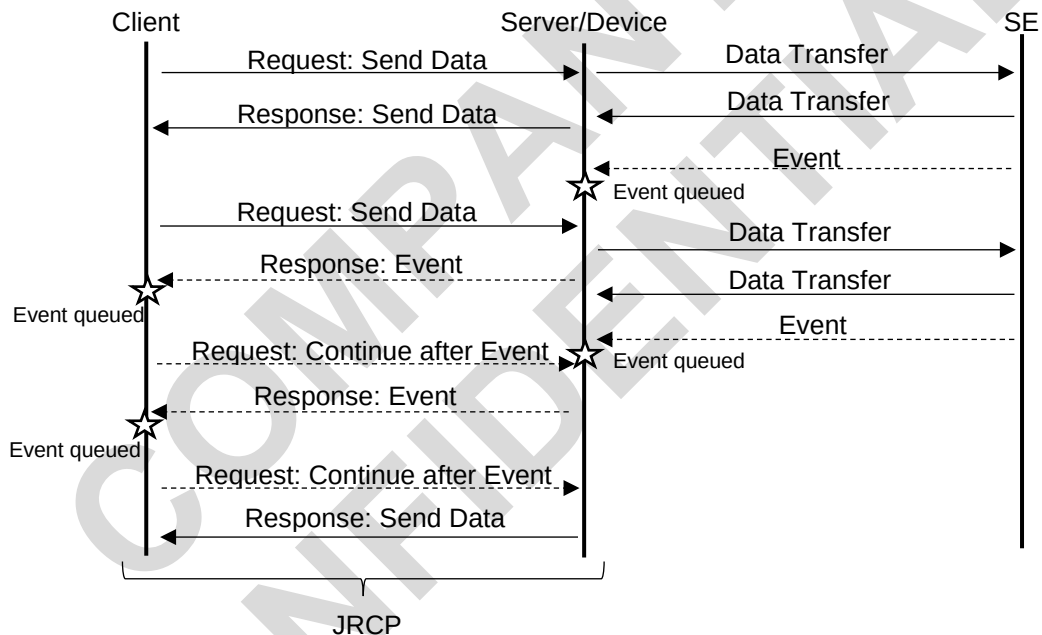


Fig 2. Communication Example including event handling

3.9 Event Handling

To handle events that might be issued in an asynchronous manner in the synchronous JRCP, events can intercept responses to commands. If this occurs, the server answers any request from the client with an event response, see Figure 2. Then, the event data is transmitted to the client instead of the intended response. Due to the event format, the client can differentiate events and responses, and re-submit the query. It is the client's responsibility to actively poll the server until the expected response is received. For this task, the "Continue After Event" request (Section 4.14) has been introduced. If this command is sent, the server responds to the previous request or handles the next event.

JCOP Remote Communication Protocol (JRCP) v2.2

On both the client and the server, queues SHALL be implemented to buffer events. The length of the queue is not specified in this document and may depend on the use case. A corresponding API SHALL exist to retrieve events from these queues. The queuing of events allows that multiple events can occur during a single transaction. Extended-length commands have an optional field to store timestamps such that events and commands can be ordered correctly.

3.10 Extended Message Format

Commands are encoded in a message format as specified in this section.

The extended message format is contained since version 2.0 of this specification.

This extended message format is defined to allow for the transmission of full-length extended APDUs, where the two bytes length field of the original message format is insufficient. This original message format is called legacy message format and is defined in section 3.11. Furthermore, additional features including support of HCI/NCI [2] has been added in the extended message format.

Table 1. Extended Message Format

Byte Number(s)	Abbreviation	Description
0	SOF	Start of message (fixed to A5h)
1	MTY	Command selection
2	NAD	Node ADdress
3	HDL	One byte length of the optional custom header ([bytes], coded as unsigned char)
4 to 4+HDL-1 (for HDL>0)	HD	Optional custom header (e.g. gate, pipe, etc., length according to value coded by HDL)
4+HDL to 7+HDL	LN	Four byte payload length ([bytes], coded as unsigned long)
8+HDL to 8+HDL+LN-1	PY	Payload (length according to value coded by LN, may include the optional sub-MTY)
8+HDL+LN	TIL	One byte coding the presence of the optional timing information (see Table 4 and Table 5)
8+HDL+LN+1 to 8+HDL+LN+8	TR	Optional, response eight bytes timestamp of the response ([ms] since start-up, coded as unsigned long long)
8+HDL+LN+9 to 8+HDL+LN+16	TC	Optional, response eight bytes timestamp of the original command ([ms] since start-up, coded as unsigned long long)

To differentiate the extended message format from the legacy message format, the first byte of an extended message format is set to A5h.

The MTY equal to A5h for the legacy message format is not allowed

The actual command, named MTY (Message Type) is encoded in the second byte of the extended message format. The payload layout of the extended format SHALL coincide with that of the standard layout for all MTYs if not specified otherwise. An optional header with variable length can be specified after the node address. For example, this can be used to specify a certain gate for SWP/HCI communication between a client and the target:

JCOP Remote Communication Protocol (JRCP) v2.2

Table 2. Example of a Request specifying a Gate and Pipe

SOF	MTY	NAD	HDL	HD	LN	Payload	TIL
A5h	01h	00h	02h	00h (gate) 00h (pipe)	00000002h	0900h	00h

Table 3. Example of a Response indicating presence of TR and TC

SOF	MTY	NAD	HDL	LN	TIL	TR	TC
A5h	01h	00h	02h	00000000h	10h	000000000010062Eh	0000000000172Eh

New server implementations SHALL not implement the legacy message format but only the extended message format. An exception is the response to the first “Terminal Information Request” that is used to initiate the communication.

Optional timestamps can be appended to the responses in form of up to two eight-byte values. Requests cannot hold timestamps. Timestamps correspond to the time in milliseconds since the start-up of the remote server or the device, if supported. If such times are made available by a device they SHALL be returned by the remote server in preference of times measured on the remote server itself. Both timing information, the TR (Timestamp Response) when the response is issued and the TC (Timestamp Command/Request) when the command has been fully received, can be added to the response. The *Timing Information Length (TIL)* byte indicates which optional timing information is returned (see Table 4 and Table 5). However, TC can only be given if preceded by TR.

Table 4. Timing Information Length coding for Requests

TIL	Description
00h	Only this value is allowed
Others	Not Allowed

Table 5. Timing Information Length coding for Responses

TIL	Description
00h	No timing information returned
08h	Timestamp response only
10h	Timestamps for both response and request
Others	Not Allowed

The maximum overall message length including headers, payload, and the optional timing information SHALL not exceed ULONG_MAX. The actual limit of a specific implementation will be typically reduced as needed and SHALL be specified in the respective documentation of the actual implementation.

JCOP Remote Communication Protocol (JRCP) v2.2

3.11 Legacy Message Format

– This message format SHALL only be used when communicating with a JRCP server that support protocol versions 0.9 or older. With the exception of the version discovery, the legacy format is considered obsolete for current implementations of this specification SHALL. –

The client sends a command as a request to the remote server and the remote server returns a response back to the client. Responses SHALL contain the same MTY value than the request. A value of 0 for the NAD is used by clients to address the target on the remote server.

Table 6. Legacy Message Format

Byte Number(s)	Abbreviation	Description
0	MTY	Command selection
1	NAD	Node ADDRESS
2 to 3	LN	Two byte payload length ([bytes], coded as unsigned short)
4	PY[0]	First byte of payload
3+LN	PY[LN]	Last byte of payload

A node address of FFh refers to the controller itself and SHALL be used for configuration. This node address identify the device and it SHALL be present in the request and in the response. The binding between node addresses and the device SHALL be stored by the controller running on the remote server and this binding is communicated to the client on request.

3.12 Sub-MTYs

Some of the later on defined commands provide more than one functionality or option of usage. To support the separation of such an optional so called sub-MTY is defined, which is transmitted in the first two bytes of the payload exchanged. If the command supports sub-MTYs, such command can be expanded with custom sub-MTYs using the reserved range sub-MTY of FF00h to FFFFh.

In cases where the request is defined to not contain a sub-MTY the according response SHALL also not contain a sub-MTY. Otherwise, in cases where the request is defined to contain a sub-MTY, the according response SHALL also contain a sub-MTY.

A successful command execution SHALL be reflected by a response containing the same sub-MTY as the request received. The reception of response containing a zero as sub-MTY indicate a failure in command indication where the response SHALL contain a status code.

Sub-MTYs, if existent in a request or response, are always counted as part of the payload meaning that the length LN SHALL include the 2 bytes size of the sub-MTY.

The sub-MTY 0000h is reserved for usage in responses and SHALL only be used to return status codes.

JCOP Remote Communication Protocol (JRCP) v2.2

3.13 Device-Specific Command Extensions

For extending functionality of specific devices beyond the contained commands and options of this specification, two main mechanisms are available:

- Device-specific commands SHALL be defined by using MTYs in the reserved range E0h to FDh. All other values of MTYs are reserved for extensions of this specification.
- For the device-specific extensions of commands specified in this document, new sub-MTYs in range FF00h to FFFFh SHALL be used.

Extensions are to be defined in a separate document referencing this document including the revision number of this document. If existing commands of this specification are extended, the MTY and corresponding sections in this document need to be referenced.

All extensions SHALL be aligned and registered with the owner of this document.

The currently registered extensions are as follows:

Table 7. Registered Device-Specific Extensions

MTY	Extension
EEh	Simulator-Specific Extensions – JRCP extensions specific to the JCOP Simulator

Note: Features of registered extensions might be integrated into future JRCP versions of future revisions of this specification.

4. Commands

Each command is identified by a unique mapping to a so called Message TYPe (MTY). All commands specified in this section SHALL be implemented by a remote server claiming support the JRCP version as defined in this document (see section 3.3.2).

If an unknown MTY is provided to the controller, it SHALL answer with the status code “unsupported command received”. In the following, only the version 2.x message format is given without a custom header and without timing information. In the legacy format, the same payload format is used and the message formats are adjusted accordingly.

MTYs in the range of D0h to FDh are reserved for device-specific extensions. These are not defined within this document and may be used to implement features available and useful to specific remote server implementations not applicable to generic remote server implementations.

4.1 Wait for Card (MTY 00h)

The *Wait for Card* command returns the *Answer to Reset* (ATR) information of the last (protocol) activation applied. Note that this command should only be called once per (protocol) activation. The (protocol) activation is triggered by a *Reset* command (see Section 4.15 or Section 4.17). If no activation sequence was performed after the connection to the NAD was opened, this command will trigger an implicit activation

JCOP Remote Communication Protocol (JRCP) v2.2

sequence. Such implicit activation sequence may trigger a reset of the connected device. If it is called a second time it may trigger another activation and that by itself may trigger another reset.

The payload contains four bytes denoting the time in milliseconds the device SHALL wait for a target being detected. The response contains the result of the target activation as payload. A response containing a zero byte payload indicate that the addressed device could not activate the target or that no target is connected. In such cases a status message or an error message command might be issued to retrieve the actual reason/failure.

If the target does not respond or the query does not match the format specification, an empty payload SHALL be returned. Further information can be received through event handling (for example, on target removal) or by querying the server status (MTY equal to 0Ah, see section 4.11).

Table 8. Wait for Card Request

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	00h	NAD	00h	00000004h	Waiting time in milliseconds	00h

Table 9. Wait for Card Response

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	00h	NAD	00h	Length	ATR of target	00h

4.2 Send Data (MTY 01h)

– This command has changed to handle APDUs for all protocols. For the legacy behavior, see Section 5.1. –

Starting with version 2.0, the data exchange of the extended message format is no longer TPDU-based, but APDU-based. As a consequence, the segmentation of APDUs into TPDUs and, in case of the T=0 protocol, the handling of the GET RESPONSE command needs to be implemented on the server side. Then, the payload is the APDU in case of requests and responses:

Table 10. Standard Send Data Request/Response

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	01h	NAD	00h	Length	Plain APDU bytes of request/response	00h

If the device does not respond, an empty payload SHALL be returned. Further information can be received through event handling (for example, on target removal) or by querying the server status (MTY 0Ah, see section 4.11).

If the tear is enabled via prepare tearing, an empty payload is returned. See Section 4.13 for details.

JCOP Remote Communication Protocol (JRCP) v2.2

4.3 Status Message (MTY 02h)

– This command is obsolete and SHALL not be used in version 2.0 and higher of the JRCP. The “Server Status Message” command (MTY 0Ah) SHALL be used for this purpose instead. –

4.4 Error Message (MTY 03h)

– This command is obsolete and SHALL not be used in version 2.0 and higher of the JRCP. The “Server Status Message” command (MTY 0Ah) SHALL be used for this purpose instead. –

4.5 Terminal Information (MTY 04h)

The “Terminal Information” command can be used to determine a human-readable string describing the connected device. The request contains a zero length payload.

Table 11. Terminal Information Request

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	04h	NAD	00h	00000000h	-	00h

For all node addresses (addressing devices) except FFh (addressing the controller), the reply contains an ASCII-encoded string specifying terminal details like device type, port, etc. The string length SHALL not exceed 1024 bytes. If the terminal information is not supported, an empty string is returned:

Table 12. Terminal Information Response

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	04h	NAD	00h	Length	Terminal Information string	00h

For JRCP version 2.0 and higher, the “Terminal Information” command is used as a first step in determining the supported JRCP version of the controller of the remote server. As a consequence, a client implementation in accordance with this specification, SHALL issue this command using the legacy message format only:

Table 13. Legacy Terminal Information Request/Response

MTY	NAD	LN	Payload
04h	NAD	Length	Information about the device

Note that the payload is identical to that of Table 11 and Table 12. For more details on the version discovery see the example communication in Section 6.1 “Version Discovery”.

For version 2.0 and later, a fixed identifier (ASCII encoded “JRCP Protocol 2+”) is returned when the controller is queried (NAD=FFh):

JCOP Remote Communication Protocol (JRCP) v2.2

Table 14. Legacy Terminal Information Response for implementations supporting JRCP version 2.0 and higher

MTY	NAD	LN	Payload
04h	FFh	0010h	4Ah 52h 43h 50h 20h 50h 72h 6Fh 74h 6Fh 63h 6Fh 6Ch 20h 32h 2Bh

To output a server-specific terminal information string, a specific command called “*Feature Control Request*” (FTY=FEh) is defined, see Section 4.22 “Feature Control Request (MTY FFh)”.

4.6 Initialization Data (MTY 05h)

– This command is obsolete and SHALL not be used in version 2.0 and higher of the JRCP. Device-specific commands (MTY FDh) SHALL be used for this purpose instead. –

4.7 Information Text (MTY 06h)

– This command is obsolete and SHALL not be used in version 2.0 and higher of the JRCP. Device-specific commands (MTY FDh) SHALL be used for this purpose instead. –

4.8 Debug Message (MTY 07h)

– This command is obsolete and SHALL not be used in version 2.0 and higher of the JRCP. Device-specific commands (MTY FDh) SHALL be used for this purpose instead. –

4.9 Change Protocol (MTY 08h)

– This command is obsolete and SHALL not be used in version 2.0 and higher of the JRCP. Device-specific commands (MTY FDh) SHALL be used for this purpose instead. –

4.10 Set Terminal Parameters (MTY 09h)

– This command is obsolete and SHALL not to be issued by clients anymore, independent of the JRCP version implemented. –

4.11 Server Status Message (MTY 0Ah)

The client can request the last status of the server (NAD=FFh) or any device, which SHALL be answered by a status code and an optional description string.

Table 15. Status Message Request

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	0Ah	NAD	00h	00000000h	-	00h

The response contains a code from section 4.23 in the payload and optionally an ASCII-encoded description string. The string length SHALL not exceed 1024 bytes.

JCOP Remote Communication Protocol (JRCP) v2.2

Table 16. Status Message Response

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	0Ah	NAD	00h	Length	Generic status code + optional ASCII encoded string	00h

4.12 Timing Information (MTY 0Bh)

The “*Timing Information*” command allows to obtain timing information from a device. This command supports sub-MTYs, which are used to configure, to query and to reset the timer.

Table 17. Timing Information Request

SOF	MTY	NAD	HDL	LN	Sub-MTY	Payload	TIL
A5h	0Bh	NAD	00h	Length	Sub-MTY	Depending on Sub-MTY	00h

The following actions are defined:

Table 18. Layout of the Payload (Action definition)

Description	Sub-MTY	Remaining bytes
Reserved for Status Codes	0000h	See Table 19 and Table 49
Reset the timer	0001h	-
Query timer	0002h	-
Set options	0003h	Four byte of options, see below
Get current options	0004h	-
Get available options	0005h	-
Not Allowed	Others	-

The response to “*Reset timer*” and “*Set options*” SHALL be one of the values in Table 19 or Table 49.

Table 19. Specific status codes for timing

Payload (after sub-MTY 0000h)	Comment
0Bh 00h	Timing OK
0Bh 01h	Timing Error

The response payload to the timer query is the measured time in microseconds.

Table 20. Timing Information Response

SOF	MTY	NAD	HDL	LN	Sub-MTY	Payload	TIL
A5h	0Bh	NAD	00h	0000000Ah	0001h	Measured time ([μ s], coded as unsigned long long)	00h

The following timing options are defined. Note that the same format is used as a response for “*Get current options*”.

JCOP Remote Communication Protocol (JRCP) v2.2

Table 21. Mask for the Supported Options

Byte 1 (reserved)	Bytes 2-3 (RFU)	Byte 4								Description
		7	6	5	4	3	2	1	0	
Reserved for device- specific extensions								X	X	1=Processing time of the reader included
										1=Transmission driver → reader included
							X			1=Transmission driver ← reader included
						X				1=Transmission terminal → driver included
					X					1=Transmission terminal ← driver included
				X						1=System time on the server side, 0=Measured on the reader/by the driver
			X							1=TR and TC requested/responded, 0=No TR and no TC
		X								RFU

The first byte of the timing options is reserved for device-specific extensions. If an option is not supported by the device, the corresponding bit is fixed to zero and not changed with the “Set options” action. This requires reading back the timing options after each change. The default options are defined to be the most accurate measurement (as close to the target as possible) available on each device.

To query whether the device supports remote timing the “Get available options” SHALL be used. The response SHALL contain one bitmask as specified above where only those bits are set to one that mark options that cannot be disabled (required options), followed by a bitmask where only those bits are set to zero that mark options that cannot be enabled (supported options).

4.13 Prepare Tearing (MTY 0Ch)

A prepare tearing command allows to enable an automated action after a given duration expired. The tear action is executed while a session with a target is active.

This might be during an APDU exchange using the “Send Data”.

Tearing is enabled using a command of MTY equal to 0Ah to set up a trigger that resets, restarts, or powers off the target after a given duration. A value of zero provided as duration SHALL disable the tearing and SHALL be accepted, by a device supporting prepare tearing, independently if such a command was sent before or not and also if a duration is remaining or did expire already.

A response with status code “Tearing OK” payload SHALL be submitted by the controller to the client. If the following “Send Data” command execution of the contained APDU finishes before the duration expires and the set event is triggered, the tearing stays enabled and triggers whenever the duration actually expires. This allows the prepare tearing command to span a sequence of commands.

The actual implementation of the duration measurement of the device depends on implementation specific. Please refer to the device documentation. This command

JCOP Remote Communication Protocol (JRCP) v2.2

supports sub-MTYs, which are used to select the action to be performed when the provided duration expires.

The first four bytes of the payload of the request defines the tearing options, directly followed by the action performed when the set duration expires:

Table 22. Tearing Preparation Request

SOF	MTY	NAD	HDL	LN	Sub-MTY	Payload	TIL
A5h	0Ch	NAD	00h	Length	Sub-MTY (see Table 23)	see Table 23	00h

The following actions are defined via sub-MTYs:

Table 23. Layout of the Payload (Action definition)

Description	Sub-MTY	Payload
Reserved for Status Codes	0000h	See Table 25 and Table 49
Reset (no power cycle)	0001h	Four bytes time information ([us], coded as unsigned long)
Reset (with power cycle)	0002h	Four bytes time information ([us], coded as unsigned long)
Not Allowed	0C00h	-
Device-specific actions	8000h – 80FFh	Four bytes time information ([us], coded as unsigned long)
Not Allowed	Others	-

Not all actions are supported by all devices. Therefore, it is essential to query the server whether the intended action is available:

Table 24. Feature Request to check if a device supports a certain tearing action (ACT)

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	FFh	FFh	00h	00000003h	0Ch NAD ACT	00h

The response SHALL be one of the values defined in Table 25 and Table 49.

The response to the prepare tearing request is either a status code of “Tearing OK” or “Tearing Error”.

Table 25. Specific status codes for tearing

Payload (after sub-MTY 0000h)	Comment
0Ch 00h	Tearing OK
0Ch 01h	Tearing Error

Not all devices support the prepare tearing command. To check for tearing support, use the “Feature Control Request” (MTY FFh) as described in Section 4.22. To achieve this, a byte corresponding to the device NAD is appended to the request. To check whether the device with node address 01h supports tearing, issue the following request:

Table 26. Feature Request to check if the device supports tearing

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	FFh	FFh	00h	00000002h	0Ch NAD	00h

JCOP Remote Communication Protocol (JRCP) v2.2

The response SHALL either be “Tearing error” if the device with NAD does not support tearing, or “Tearing OK”, otherwise. Both are defined in Section 4.11 “Status Messages” (MTY 0Ah). Tearing preparation requests SHALL not be issued if the device does not support tearing.

4.14 Event Handling (MTY 0Dh)

Asynchronous events may be emitted by a device. As described above, these events intercept the response to a request and are sent to the client instead. The client can poll for events using the following format:

Table 27. Event Request

SOF	MTY	NAD	HDL	LN	Sub-MTY	Payload	TIL
A5h	0Dh	NAD	00h	00000002h	Sub-MTY	-	00h

The sub-MTY for *requests* may be one of the following values:

Table 28. Sub-MTYs for Event Requests

Sub-MTY	Event Type
0000h	Request any event
0001h	Request only non-HCI events
0002h	Request only HCI events
Others	Not Allowed

The response format and general format of the events is defined as follows:

Table 29. Event Response

SOF	MTY	NAD	HDL	LN	Sub-MTY	Payload	TIL
A5h	0Dh	NAD	00h	Length	sub-MTY	Payload	00h

In this case, the sub-MTYs determine the event type and payload:

Table 30. Sub-MTYs for Event Responses and Corresponding Payload

Sub-MTY	Payload	Event Type
0000h	Status code (see Section 4.23 “Generic Status Codes”)	Status Code
0001h	Raw event data	Non-HCI event
0002h	Raw event data	HCI events
Others	Not Allowed	

In case no event is queued, the status code “Status OK” (00h 00h) is returned:

Table 31. Event Response if no Event Present

SOF	MTY	NAD	HDL	LN	Sub-MTY	Payload	TIL
A5h	0Dh	NAD	00h	00000004h	0000h	0000h (“Status OK”)	00h

JCOP Remote Communication Protocol (JRCP) v2.2

To continue with processing the last command, the “Event Request” command with any sub-MTY SHALL be issued by the client. The next response from the remote server SHALL be as if the last command had been sent directly. If the previous command (or the continuation request) is issued in the extended-length format, the timestamp of the event issuance SHALL be present in the event response.

For HCI events, the same framework SHALL be used with the sub-MTY 00h 02h. It is specified as follows:

Table 32. HCI Get Event Request

SOF	MTY	NAD	HDL	LN	Sub-MTY	Payload	TIL
A5h	0Dh	NAD	00h	00000002h	0002h	-	00h

The response SHALL be of the following format:

Table 33. HCI Get Event Response

SOF	MTY	NAD	HDL	Gate ID	Pipe ID	Event type	LN	Sub-MTY	Payload	TIL
A5h	0Dh	NAD	03h	G	P	ET	Length	0002h	Raw event data	00h

If HCI events are queued on the server, the gate ID (G) of the event, as well as its pipe (P) and event type (ET) SHALL be included in the header. The actual event data in the payload depends on the event type.

4.15 Set IO Pin (MTY 0Eh)

This command allows to set a particular IO pin on the device to a given value. The request format is as follows:

Table 34. Set IO Pin Request

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	0Eh	NAD	00h	00000002h	ID+VAL	00h

Both the pin identifier ID and the given value VAL are represented by a one byte value. VAL may either be 00h or 01h depending if the IO SHALL be cleared or set. Please refer to the user manual of the device for allowed values of ID.

This feature may not be supported by all devices. Therefore, it is essential to query the server whether this feature is available:

Table 35. Feature Request to check if the device supports a set IO pin action (ACT)

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	FFh	FFh	00h	00000003h	0Eh+NAD+ACT	00h

The response SHALL either be “OK” or “Generic Error” as defined in Section 4.23 “Generic Status Codes”.

The response to the set IO pin SHALL be the value read back from the pin after setting it on the remote side. It is up to the client to check whether the returned value matches the intended one.

JCOP Remote Communication Protocol (JRCP) v2.2

In case an error occurred, a status message “Generic Error” SHALL be issued, see Section 4.23 “Generic Status Codes”. The message of the last error SHALL be queried using “Error Message (MTY 03h)”, cf. Section 4.4.

4.16 Warm Reset (MTY 30h)

To reset a target from remote, a *warm reset* can be requested by issuing a command to the device. A waiting time before the reset is actually triggered is specified in the payload of the request.

Table 36. Warm Reset Request

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	30h	NAD	00h	00000004h	Waiting time in milliseconds	00h

The ATR following the reset is returned to the sender in the payload.

Table 37. Warm Reset Response

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	30h	NAD	00h	Length	ATR of target	00h

If the target does not respond or the query does not match the format specification, an empty payload SHALL be returned by the addressed device. Further information can be received through event handling (for example, on target removal) or by querying the server status, see Section 4.11 “Server Status Message (MTY 0Ah)”.

4.17 Cold Reset (MTY 31h)

To reset a target from remote, a *cold reset* can be requested by issuing a command to the device. A waiting time before the reset is actually triggered is specified in the payload.

Table 38. Cold Reset Request

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	31h	NAD	00h	00000004h	Waiting time in milliseconds	00h

The ATR following the reset is returned to the sender in the payload.

Table 39. Cold Reset Response

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	31h	NAD	00h	Length	ATR of remote target	00h

If the target does not respond or the query does not match the format specification, an empty payload SHALL be returned by the addressed device. Further information can be received through event handling (for example, on target removal) or by querying the server status, see Section 4.11 “Server Status Message (MTY 0Ah)”.

4.18 HCI Send Data (MTY 32h)

HCI communication is split into two cases: sending of HCI events to the device and retrieving events from it. The latter is handled as part of Section 4.14 “Event Handling (MTY

JCOP Remote Communication Protocol (JRCP) v2.2

0Dh)". Note that not all devices support HCI, so it is mandatory to check whether this command is supported by the device using the "Feature Control Request" (MTY FFh) as described in Section 4.22. Sending HCI events requires an additional header to specify the HCI Gate (G) and event type (ET):

Table 40. HCI Send Event Request

SOF	MTY	NAD	HDL	Gate ID	Event type	Frame type	LN	Payload	TIL
A5h	32h	NAD	03h	G	ET	FT	Length	Event Data	00h

Here, G specifies the Gate ID with allowed value as specified in [3] and are given in the following table.

Table 41. HCI Gate IDs

Gate	Value
Loop back gate	04h
Type B card RF gate	21h
Type B' card RF gate	22h
Type A card RF gate	23h
Type F card RF gate	24h
ISO/IEC 14443 Type B reader RF gate	11h
ISO/IEC 14443 Type A reader RF gate	13h
Connectivity gate	41h
APDU gate	30h

The value for the event type (ET) matches the specification of [3]. For a full list of possible event types and the corresponding values, please refer to that document.

Frame type (FT) accepts the following values:

Table 42. Frame Types

Frame type	Value
CLT Frame	01h
IFrame	02h

The response to "HCI Send Event" is in the following format:

Table 43. HCI Send Event Response

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	32h	NAD	00h	00000002h	Status code	00h

The following status codes are defined:

Table 44. Specific status codes for HCI Send Event

Payload	Comment
32h 00h	HCI Send Event OK

JCOP Remote Communication Protocol (JRCP) v2.2

Payload	Comment
32h 01h	HCI Send Event Error

4.19 Reserved for Extended Commands (MTY A5h)

– This MTY is reserved in the standard layout to indicate extended-length commands. –

4.20 Device-specific Extensions (MTY E0h to FDh)

This range of MTYs is reserved for device-specific extensions. If no extensions are available, all requests having this MTY SHALL be answered with the status message “Generic Error”, see Section 4.23 “Generic Status Codes”.

In order to check which device is connected at an NAD, the “Terminal Information” (MTY 04h) command described in Section 4.5 SHALL be used. Based on this, device-specific extensions can be discovered using “Feature Control Requests”, cf. Section 4.22. Before particular extensions are queried, a type a request SHALL be performed in order to check whether the device implements any device-specific extensions. This SHALL return the version defined in the specification of that particular extension as a two byte value and SHALL identify the feature set of the extension in accordance to the device-specific extensions specification.

4.21 Controller Configuration (MTY FEh)

This section defined the command and available options to configure the controller. New MTYs SHALL only be assigned to operations on/with devices, but not the controller itself. The generic request and response format looks like:

Feature Controller Configuration Request/Response

SOF	MTY	NAD	HDL	LN	Payload	TIL
A5h	FEh	FFh	00h	Length	2 Bytes sub-MTY + optional payload/response	00h

The first two byte of the payload (sub-MTY) constitute the configuration type. The sub-MTY SHALL be repeated in the response. It may be one of:

Table 45. Supported Configuration Types (sub-MTYs)

sub-MTY	Comment
0000h	Reserved for status codes, see Table 49
0001h	Request list of connected devices. On return, a TLV encoded list of their names (as ASCII encoded strings) and their mapping to NADs is returned. Tag and length are encoded in one byte each. If no device is present, an empty string is returned. The string length SHALL not exceed 255 bytes per device. NADs in the range 00h – 7Fh denote nodes that are predefined by the controller, NADs between 80h – FEh have been assigned by clients. For devices present on the remote side but not yet mapped to a node address “FFh” SHALL be chosen. The type of the NAD is 00h, for the reader name 01h SHALL be chosen.

JCOP Remote Communication Protocol (JRCP) v2.2

sub-MTY	Comment
0002h	Freeze current mapping between remote devices and NADs. A controller may not support this command if only pre-defined nodes are allowed. Then, the generic status code "Unsupported Command" SHALL be returned, see Section 4.23 "Generic Status Codes".
0003h	Release mapping between remote devices and NADs. This SHALL cause the controller to close all open connections to attached devices. A controller may not support this command if only pre-defined nodes are allowed. Then, the generic status code "Unsupported Command" SHALL be returned, see Section 4.23 "Generic Status Codes".
0004h	Connect to the specified terminal. The request contains an ASCII encoded string of the connection parameters interpreted by the controller. For a JRCP server based on nxp-offcard/JCShell a terminal string as used in JCShell SHALL be provided, e.g. "SCComm:1". The string length SHALL not exceed 1024 bytes. The response is either the status message the assigned NAD in case of a successful connection, or "Generic Error" otherwise. A controller may not support this command if the requested node is a pre-defined one. Then, the generic status code "Unsupported Command" SHALL be returned, see Section 4.23 "Generic Status Codes".
0005h	Close a terminal attached at the specified NAD. The NAD is given in the payload of the request. The controller response with the status message "Acknowledge/OK" or "Generic Error". A controller may not support this command if the requested node is a pre-defined one. Then, the generic status code "Unsupported Command" SHALL be returned, see Section 4.23 "Generic Status Codes".

In case an error occurred, a status message "Generic Error" SHALL be issued, see Section 4.11 "Server Status Message (MTY 0Ah)". The message of the last error SHALL be queried using "Error Message (MTY 03h)", cf. Section 4.4.

4.22 Feature Control Request (MTY FFh)

This command is used to query the implemented JRCP version of the controller running on the remote server. The query type is indicated by the Feature TYpe (FTY) byte, which is encoded in the first byte of the request payload. The generic feature control format is defined in the following table for the request format followed by a table defining the response format:

Table 46. Feature Control Request

SOF	MTY	NAD	HDL	LN	Sub-MTY	Payload	TIL
A5h	FFh	NAD	00h	Length	Sub-MTY	One byte FTY + optional payload	00h

Table 47. Feature Control Response

SOF	MTY	NAD	HDL	LN	Sub-MTY	Payload	TIL
A5h	FFh	NAD	00h	Length	Sub-MTY	Optional payload (see Table 48)	00h

In general, requests are addressed to the server (NAD=FFh) to requests its capabilities. Certain operations such as tearing preparation commands are not available on all devices. Support for these operations on a specific device can be requested by specifying the corresponding NAD.

JCOP Remote Communication Protocol (JRCP) v2.2

The following sub-MTYs are supported:

Table 48. Supported Request Types (sub-MTYs)

sub-MTY	Comment
0000h	Reserved for status codes, see Table 49
0001h	Version control/request whether a sub-MTY is supported: the payload of the query command consists of one byte FTY only. When the MTY corresponding to the FTY is supported by the implementation the controller returns a two byte long payload containing its JRCP version, as defined in section 3.3.2, where the first byte SHALL contain the major version and the second byte SHALL contain the minor version. In case of device-specific extensions, the version of the respective extension SHALL be returned. When the MTY is not supported a payload of "0000h" SHALL be returned. MTYs not specified in this document SHALL also result "0000h".
0002h	Optional check for feature options: additional payload can be appended to the FTY sub-MTY to query specific functionality (e.g. particular sub-command) within the corresponding MTY. This request type is optional and is defined for each MTY in its corresponding section.
0003h	Query an ASCII encoded identifier of the controller. The returned identifier may be explicitly specified by a device-specific implementation to identify that extension uniquely. The returned identifier string SHALL not be an empty string (zero length string) and the string is not NULL terminated (the caller SHALL not rely on the NULL termination to be present in the response). The string length SHALL not exceed 1024 bytes.
0004h	Return the JRCP version implemented by the controller as a two byte value. The first byte corresponds to the major version, the second one to the minor version. An implementation according to this specification SHALL return a value as defined in section 3.3.2. Example: version "2.0" SHALL be returned as "02h 00h".
0005h	This sub-MTY followed by a 16 byte payload allows to submit a session ID to the controller. This ID may be used by the controller for mapping log files to a particular sessions. The response to this command SHALL either be "Acknowledge/OK" or "Unsupported command received", see Section 4.23 "Generic Status Codes".

A controller running on a remote server returning a particular version of the JRCP as a response to sub-MTY 0004h SHALL implement the complete protocol of as defined in the specification of that version. Though, some commands might support a more recent version, which is then indicated in the response to sub-MTY 0001h.

4.23 Generic Status Codes

These codes can be used in the payload of responses.

Table 49. Generic Status codes

Payload (after sub-MTY 0000h)	Comment
0000h	Acknowledge/OK
0001h	Generic Error
0002h	Target removed
00FFh	Unsupported command received
Any other value	Other Error

JCOP Remote Communication Protocol (JRCP) v2.2

Additional, specific status codes may be defined as part of device-specific extensions. However, any status code in the range of any other value defined outside of this specification SHALL only code details about non successful command execution and SHALL be interpreted as an error for implementations based on this specification. Any successful command execution SHALL always be indicated by 0000h only.

5. Obsolete Commands

The following are obsolete starting with JRCP version 2.0 and above and are listed for completeness. They are deprecated and SHALL not be supported in new implementations anymore. If a corresponding command is received at the server, either in the new or the legacy message format, a status message "Unsupported command received" (Section 4.11) SHALL be returned.

5.1 Send Data (MTY 01h)

For commands using the legacy message format, *Send Data* sends and receives unsegmented TPDUs in the payload. In case of the T=1 and T=CL protocols, this is equivalent to APDUs for both the request and the response. For T=0 requests, containing Command APDUs (C-APDUs), may need to be split using ENVELOP commands and responses, containing Response APDUs (R-APDUs) needs be reconstructed using GET RESPONSE commands. The legacy request and response format is:

Table 50. Legacy Send Data Request/Response Format

MTY	NAD	LN	Payload
01h	NAD Value	Length	Unsegmented TPDUs bytes of request/response

Due to the limitation of the coding of the payload length, the unsegmented TPDUs including the header may be at most 65,535 bytes long in the legacy format.

5.2 Status Message (MTY 02h)

Status Message requests the current status of the remote device. The request contains no payload.

Table 51. Status Message Request Format

MTY	NAD	LN	Payload
02h	NAD Value	0000h	-

The response of the remote device is encoded in a fixed length of 4 bytes.

Table 52. Status Message Response Format

MTY	NAD	LN	Payload
02h	NAD Value	0004h	Status of the remote device

JCOP Remote Communication Protocol (JRCP) v2.2

5.3 Initialization Data (MTY 05h)

Initialization Data sends a set of data which SHALL be used by the remote device to initialize itself. The interpretation of this data is up to the remote device.

Table 53. Initialization Data Request Format

MTY	NAD	LN	Payload
05h	NAD Value	Length	Initialization data

In case of an error during initialization, the response SHALL contain an error description in the payload. Otherwise no payload SHALL be sent.

Table 54. Initialization Data Response Format

MTY	NAD	LN	Payload
05h	NAD Value	Length	Payload only in case of errors

5.4 Information Text (MTY 06h)

Information Text sends a message to the remote device for displaying to the operator or to be used for logging purposes. The data SHALL be a string encoded according to ISO8859-1.

Table 55. Information Text Request Format

MTY	NAD	LN	Payload
06h	NAD Value	Length	Information text

In case of errors, the response SHALL contain an error description in the payload. Otherwise no payload SHALL be sent.

Table 56. Information Text Response Format

MTY	NAD	LN	Payload
06h	NAD Value	Length	Payload only in case of errors

5.5 Debug Message (MTY 07h)

Debug Message allows sending debug commands to the remote device. The debug command is contained in the payload. The format is specific the remote device.

Table 57. Debug Message Request Format

MTY	NAD	LN	Payload
07h	NAD Value	Length	Debug message

JCOP Remote Communication Protocol (JRCP) v2.2

The response payload is optional and may contain debug information from the remote device.

Table 58. Debug Message Response Format

MTY	NAD	LN	Payload
07h	NAD Value	Length	Debug information from the remote device

5.6 Change Protocol (MTY 08h)

– This MTY is only available on the UART interface of the JCOP Simulator. Therefore, it has been moved into the device-specific extensions of the JRCP (for protocol versions > 2.0). –

This command allows to change the protocol of the APDU exchange on a UART interface. Issuing the command on any other interface results in an error message.

Table 59. Change Protocol Request Format

MTY	NAD	LN	Payload
08h	NAD Value	0001h	Higher four bits: media type Lower four bits: protocol specifier

The media type supports the following values:

Table 60. Values for the media type

Media type	Comment
00h	Contact mode
08h	Contactless, type A mode
09h	Contactless, type B mode

For the protocol specifier, the following values might be chosen:

Table 61. Values for the protocol specifier

Protocol specifier	Comment
00h	Change protocol to T=0
01h	Change protocol to T=1

5.7 Set Terminal Parameters (MTY 09h)

– Obsolete feature. –

JCOP Remote Communication Protocol (JRCP) v2.2

6. Examples

Examples for the communication between a client and a target using JRCP are illustrated in this section. As a client, a JCSHELL is assumed that connects to a remote server. On the remote server a controller implements a connection to a device at node address (NAD) 00h that itself interfaces to a JCOP enabled product. The examples consist of a version discovery, a target activation and command exchange as well as an example of an HCI event exchange.

6.1 Version Discovery

The initial version discovery requires a query for the terminal information of the controller running on the remote server. Therefore, the initial communication between client and server SHALL be:

Table 62. Example: Version Discovery

Client	Direction	Device
Get terminal information from the server		
04 FF 00 00	→	
	←	04 FF 00 10 4A 52 43 50 20 50 72 6F 74 6F 63 6F 6C 20 32 2B
<i>Version 2.0+ detected, continue with extended format, otherwise abort</i>		
Get protocol version		
A5 FF FF 00 00 00 00 02 00 04 00	→	
	←	A5 FF FF 00 00 00 00 04 00 04 02 00 00

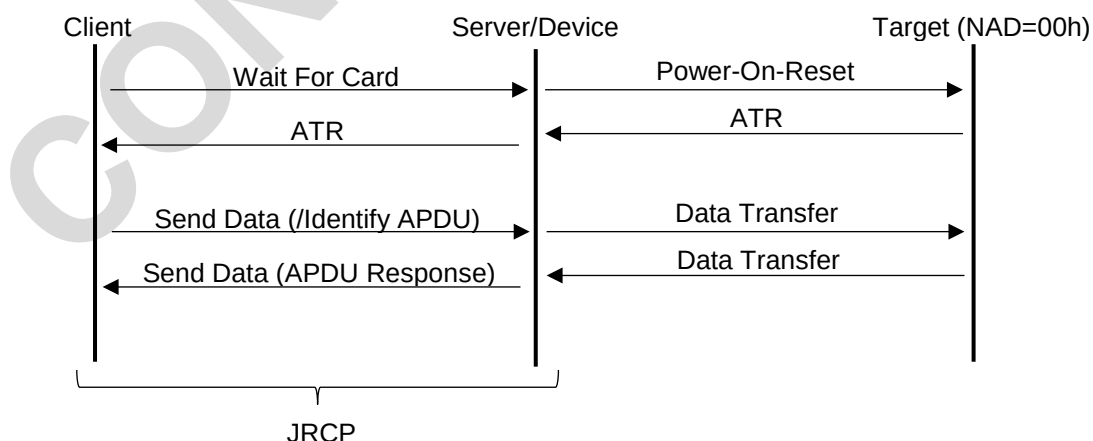


Fig 3. Sequence diagram: Version Discovery

JCOP Remote Communication Protocol (JRCP) v2.2

Note: All values listed in Table 62 are given in hexadecimal format, for readability the suffix 'h' is omitted.

Note: This is the only use case in which the legacy message format may be used in the scope of implementations of JRCP versions 2.0 or higher versions.

Note: The "Get Protocol Version" command is described in Section 4.22.

6.2 Target Activation and Command Exchange

The communication sequence is shown in Fig. 3: after an initial "Wait for Card", an "IIdentify" command is sent to the target the "Send Data" command. The controller forwards the request to the addressed device, which processes this request, applies the needed actions for the connected target and returns information collected during the boot process of the target in the response to the controller, which forwards it to the client via JRCP. In a second step, an information text is sent to the server (NAD=FFh) to be logged there. Typically, these messages are used to indicate the start and stop of a test case to be performed on the remote device. The server then processes the information text and responds with the status message "Acknowledge/OK".

The commands exchanged in this example communication are given in **Error! Reference source not found..**

Table 63. Example: Target Activation and Command Exchange

Client	Direction	Device
Wait for Card (5000ms)		
A5 00 01 00 00 00 00 04 00 00 13 88 00	→	
	←	A5 00 01 00 00 00 00 12 3B F8 13 00 00 81 31 FE 45 4A 43 4F 50 76 32 34 31 B7 00
Send Data (IIdentify)		
A5 01 01 00 00 00 00 0E 00 A4 04 00 09 A0 00 00 01 67 41 30 00 FF 00	→	
	←	A5 01 00 00 00 00 00 12 00 00 01 00 00 00 00 00 74 6F 6F 6C 73 20 00 00 6A 82 00

Note: All values listed in this table are given in hexadecimal format, for readability the suffix 'h' is omitted.

JCOP Remote Communication Protocol (JRCP) v2.2

6.3 HCI Event Exchange

Figure 4 illustrates the basic communication sequence between a client and server for HCI events. Initially the JRCP version is determined and HCI events are sent to the server using “HCI Send Event” (MTY 32h). To retrieve events from the device (stored on the server), the “HCI Get Events” (MTY 33h) is used. As the devices typically have a limited queue for HCI events, this polling must be repeated at regular intervals.

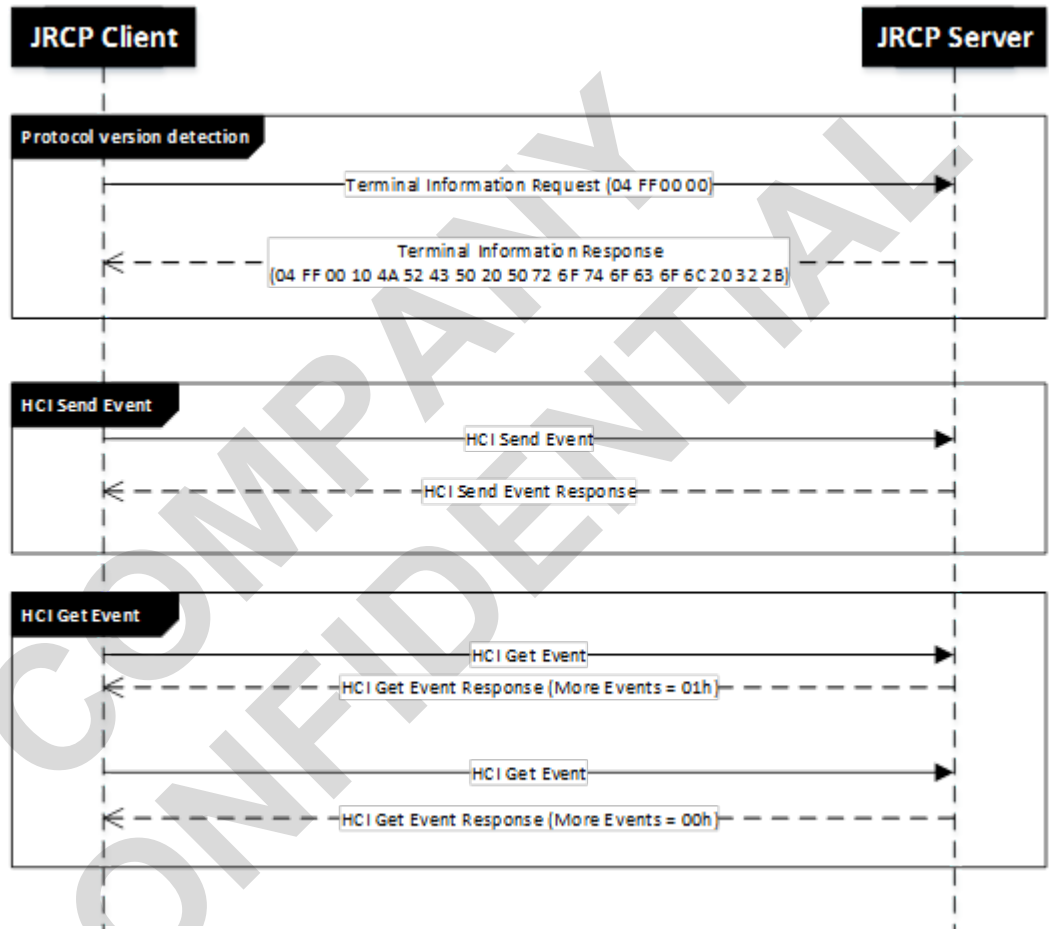


Fig 4. Sequence diagram: HCI Event Exchange

7. Legal information

7.1 Disclaimers

Limited warranty and liability — Information in this document is believed to be accurate and reliable. However, NXP Semiconductors does not give any representations or warranties, expressed or implied, as to the accuracy or completeness of such information and SHALL have no liability for the consequences of use of such information.

In no event SHALL NXP Semiconductors be liable for any indirect, incidental, punitive, special or consequential damages (including - without limitation - lost profits, lost savings, business interruption, costs related to the removal or replacement of any products or rework charges) whether or not such damages are based on tort (including negligence), warranty, breach of contract or any other legal theory.

Notwithstanding any damages that customer might incur for any reason whatsoever, NXP Semiconductors' aggregate and cumulative liability towards customer for the products described herein SHALL be limited in accordance with the Terms and conditions of commercial sale of NXP Semiconductors.

Right to make changes — NXP Semiconductors reserves the right to make changes to information published in this document, including without limitation specifications and product descriptions, at any time and without notice. This document supersedes and replaces all information supplied prior to the publication hereof.

Suitability for use — NXP Semiconductors products are not designed, authorized or warranted to be suitable for use in life support, life-critical or safety-critical systems or equipment, nor in applications where failure or malfunction of an NXP Semiconductors product can reasonably be expected to result in personal injury, death or severe property or environmental damage. NXP Semiconductors accepts no liability for inclusion and/or use of NXP Semiconductors products in such equipment or applications and therefore such inclusion and/or use is at the customer's own risk.

Applications — Applications that are described herein for any of these products are for illustrative purposes only. NXP Semiconductors makes no representation or warranty that such applications will be suitable for the specified use without further testing or modification.

Customers are responsible for the design and operation of their applications and products using NXP Semiconductors products, and NXP Semiconductors accepts no liability for any assistance with applications or customer product design. It is customer's sole responsibility to determine whether the NXP Semiconductors product is suitable and fit for the customer's applications and

products planned, as well as for the planned application and use of customer's third party customer(s). Customers should provide appropriate design and operating safeguards to minimize the risks associated with their applications and products.

NXP Semiconductors does not accept any liability related to any default, damage, costs or problem which is based on any weakness or default in the customer's applications or products, or the application or use by customer's third party customer(s). Customer is responsible for doing all necessary testing for the customer's applications and products using NXP Semiconductors products in order to avoid a default of the applications and the products or of the application or use by customer's third party customer(s). NXP does not accept any liability in this respect.

Export control — This document as well as the item(s) described herein may be subject to export control regulations. Export might require a prior authorization from national authorities.

Evaluation products — This product is provided on an "as is" and "with all faults" basis for evaluation purposes only. NXP Semiconductors, its affiliates and their suppliers expressly disclaim all warranties, whether express, implied or statutory, including but not limited to the implied warranties of non-infringement, merchantability and fitness for a particular purpose. The entire risk as to the quality, or arising out of the use or performance, of this product remains with customer.

In no event SHALL NXP Semiconductors, its affiliates or their suppliers be liable to customer for any special, indirect, consequential, punitive or incidental damages (including without limitation damages for loss of business, business interruption, loss of use, loss of data or information, and the like) arising out of the use of or inability to use the product, whether or not based on tort (including negligence), strict liability, breach of contract, breach of warranty or any other theory, even if advised of the possibility of such damages.

Notwithstanding any damages that customer might incur for any reason whatsoever (including without limitation, all damages referenced above and all direct or general damages), the entire liability of NXP Semiconductors, its affiliates and their suppliers and customer's exclusive remedy for all of the foregoing SHALL be limited to actual damages incurred by customer based on reasonable reliance up to the greater of the amount actually paid by customer for the product or five dollars (US\$5.00). The foregoing limitations, exclusions and disclaimers SHALL apply to the maximum extent permitted by applicable law, even if any remedy fails of its essential purpose.

8. References

- [1] S. Bradner, *RFC2119: Key words for use in RFCs to Indicate Requirement Levels*, 1997.
- [2] NFC Forum™, *NFC Controller Interface (NCI) Specification: NCI 1.0*, 2012.
- [3] ETSI, Technical Specification TS 102 622 V12.1.0, Sophia Antipolis Cedex, 2014.

9. List of figures

Fig 1.	Client and Server Device Architecture	5
Fig 2.	Communication Example including event handling	9
Fig 3.	Sequence diagram: Version Discovery	30
Fig 4.	Sequence diagram: HCI Event Exchange.....	32

COMPANY
CONFIDENTIAL

JCOP Remote Communication Protocol (JRCP) v2.2

10. List of tables

Table 1.	Extended Message Format.....	10
Table 2.	Example of a Request specifying a Gate and Pipe.....	11
Table 3.	Example of a Response indicating presence of TR and TC	11
Table 4.	Timing Information Length coding for Requests	11
Table 5.	Timing Information Length coding for Responses	11
Table 6.	Legacy Message Format.....	12
Table 7.	Wait for Card Request	14
Table 8.	Wait for Card Response.....	14
Table 9.	Standard Send Data Request/Response	14
Table 10.	Terminal Information Request.....	15
Table 11.	Terminal Information Response	15
Table 12.	Legacy Terminal Information Request/Response	15
Table 13.	Legacy Terminal Information Response for implementations supporting JRCP version 2.0 and higher	16
Table 14.	Status Message Request.....	16
Table 15.	Status Message Response	17
Table 16.	Timing Information Request.....	17
Table 17.	Layout of the Payload (Action definition).....	17
Table 18.	Specific status codes for timing.....	17
Table 19.	Timing Information Response	17
Table 20.	Mask for the Supported Options	18
Table 21.	Tearing Preparation Request.....	19
Table 22.	Layout of the Payload (Action definition).....	19
Table 23.	Feature Request to check if a device supports a certain tearing action (ACT)	19
Table 24.	Specific status codes for tearing	19
Table 25.	Feature Request to check if the device supports tearing.....	19
Table 26.	Event Request	20
Table 27.	Sub-MTYs for Event Requests.....	20
Table 28.	Event Response.....	20
Table 29.	Sub-MTYs for Event Responses and Corresponding Payload	20
Table 30.	Event Response if no Event Present	20
Table 31.	HCI Get Event Request	21
Table 32.	HCI Get Event Response.....	21
Table 33.	Set IO Pin Request	21
Table 34.	Feature Request to check if the device supports a set IO pin action (ACT).....	21
Table 35.	Warm Reset Request.....	22
Table 36.	Warm Reset Response	22
Table 37.	Cold Reset Request	22
Table 38.	Cold Reset Response	22
Table 39.	HCI Send Event Request	23
Table 40.	HCI Gate IDs.....	23
Table 41.	Frame Types	23
Table 42.	HCI Send Event Response	23
Table 43.	Specific status codes for HCI Send Event.....	23
	Feature Controller Configuration Request/Response.....	24
Table 44.	Supported Configuration Types (sub-MTYs) ...	24
Table 45.	Feature Control Request.....	25
Table 46.	Feature Control Response	25
Table 47.	Supported Request Types (sub-MTYs)	26
Table 48.	Generic Status codes.....	26
Table 49.	Legacy Send Data Request/Response Format	27
Table 50.	Status Message Request Format.....	27
Table 51.	Status Message Response Format.....	27
Table 52.	Initialization Data Request Format	28
Table 53.	Initialization Data Response Format	28
Table 54.	Information Text Request Format.....	28
Table 55.	Information Text Response Format.....	28
Table 56.	Debug Message Request Format	28
Table 57.	Debug Message Response Format.....	29
Table 58.	Change Protocol Request Format.....	29
Table 59.	Values for the media type.....	29
Table 60.	Values for the protocol specifier	29
Table 61.	Example: Version Discovery	30
Table 62.	Example: Target Activation and Command Exchange	31

JCOP Remote Communication Protocol (JRCP) v2.2

NXP Semiconductors

Protocol Specification

11. Contents

1.	Introduction	3			
1.1	Overview and Scope	3	4.10	Set Terminal Parameters (MTY 09h)	16
1.2	Intended Audience	3	4.11	Server Status Message (MTY 0Ah)	16
1.3	Ordering JCOP Tools	3	4.12	Timing Information (MTY 0Bh)	17
1.4	Nomenclature	3	4.13	Prepare Tearing (MTY 0Ch)	18
1.5	Notation	4	4.14	Event Handling (MTY 0Dh)	20
1.6	Reserved for Future Use	4	4.15	Set IO Pin (MTY 0Eh)	21
1.7	Not Allowed	4	4.16	Warm Reset (MTY 30h)	22
2.	JRCP Overview	5	4.17	Cold Reset (MTY 31h)	22
3.	JRCP Definition	5	4.18	HCI Send Data (MTY 32h)	22
3.1	Data types and value ranges	6	4.19	Reserved for Extended Commands (MTY A5h)	24
3.2	Endianness	6	4.20	Device-specific Extensions (MTY E0h to FDh)	24
3.3	Version Discovery	6	4.21	Controller Configuration (MTY FEh)	24
3.3.1	Version Agreement	7	4.22	Feature Control Request (MTY FFh)	25
3.3.2	JRCP Version	7	4.23	Generic Status Codes	26
3.4	Server Configuration	8	5.	Obsolete Commands	27
3.5	Device Binding	8	5.1	Send Data (MTY 01h)	27
3.6	Target Activation	8	5.2	Status Message (MTY 02h)	27
3.7	Command Exchange	8	5.3	Initialization Data (MTY 05h)	28
3.8	Mapping Between Devices and NADs	8	5.4	Information Text (MTY 06h)	28
3.9	Event Handling	9	5.5	Debug Message (MTY 07h)	28
3.10	Extended Message Format	10	5.6	Change Protocol (MTY 08h)	29
3.11	Legacy Message Format	12	5.7	Set Terminal Parameters (MTY 09h)	29
3.12	Sub-MTYs	12	6.	Examples	30
3.13	Device-Specific Command Extensions	13	6.1	Version Discovery	30
4.	Commands	13	6.2	Target Activation and Command Exchange	31
4.1	Wait for Card (MTY 00h)	13	6.3	HCI Event Exchange	32
4.2	Send Data (MTY 01h)	14	7.	Legal information	33
4.3	Status Message (MTY 02h)	15	7.1	Disclaimers	33
4.4	Error Message (MTY 03h)	15	8.	References	34
4.5	Terminal Information (MTY 04h)	15	9.	List of figures	34
4.6	Initialization Data (MTY 05h)	16	10.	List of tables	35
4.7	Information Text (MTY 06h)	16	11.	Contents	36
4.8	Debug Message (MTY 07h)	16			
4.9	Change Protocol (MTY 08h)	16			

Please be aware that important notices concerning this document and the product(s) described herein, have been included in the section 'Legal information'.

© NXP B.V. 2017.

All rights reserved.

For more information, visit: <http://www.nxp.com>
For sales office addresses, please send an email to: salesaddresses@nxp.com

Date of release: Jul 21, 2017

Document identifier: